

NETRA

Persistent Geospatial Intelligence & Autonomous Change Detection

Feature Specification, Problem Definition & Implementation Blueprint

Core concept: Netra does not merely answer questions about satellite imagery. It maintains a historical spatial memory of geographic areas, detects meaningful changes between observations, identifies potential anomalies, and alerts the user for human verification.

Project: NETRA — Geospatial Intelligence System
Technical Design Document

1. Executive Summary

The Persistent Geospatial Intelligence Layer (PGIL) is a core Netra subsystem designed to transform individual satellite-image interactions into a persistent, time-aware intelligence system. Every uploaded scene becomes a geospatial observation containing its location, acquisition time, sensor metadata, detected objects, geometries, statistics, and analytical results.

When a later observation covers the same geographic area, Netra retrieves the historical state and performs temporal and spatial comparison in the background. This comparison is independent of the user's immediate question. Therefore, a user can ask a completely unrelated question while Netra simultaneously detects potential changes and raises an alert.

Design principle: Observe → Remember → Compare → Detect → Alert → Investigate.

2. Problem Description

Most satellite-image AI systems are stateless and query-driven. A user uploads an image, asks a question, receives an answer, and the observation is effectively forgotten from the perspective of future analysis. Even when a later image of the same location is uploaded, the system may not know what existed previously, where objects were located, or whether a new observation represents meaningful change.

This limitation is significant for urban monitoring, infrastructure development, environmental monitoring, flood assessment, deforestation, agricultural monitoring, construction tracking, and other applications where the most valuable information is often the **change over time**, not the state at a single moment.

Problem to solve: How can an AI system maintain a persistent spatial memory of geographic areas and automatically identify meaningful changes or anomalies between satellite observations over time, even when the user does not explicitly request change detection?

3. Proposed Feature

Netra introduces a persistent geospatial memory in which each geographic area has an identity and a sequence of historical observations. Each observation is processed into structured geographic information such as building footprints, road networks, water bodies, vegetation statistics, change masks, confidence values, and other detected features.

A new observation is matched to a previously known geographic area using its geospatial footprint. Netra then performs registration, change detection, object-level comparison, anomaly scoring, and optional multi-sensor verification. If the result crosses configured significance thresholds, an alert is created.

4. Example User Scenario

Observation 1 — January 2026: The user uploads an image and asks, “How many buildings are there?” Netra detects 247 buildings and stores their geographic footprints as part of AREA_001.

Observation 2 — July 2026: The user uploads new imagery of the same area and asks, “Identify the water bodies.” Netra answers the water-body query normally. In parallel, its intelligence path retrieves the January observation and compares it with the July observation.

Suppose the new analysis detects 253 buildings. Netra identifies six potentially new building structures and generates an alert such as: “**Potential urban development detected — 6 new building structures identified since January 2026.**”

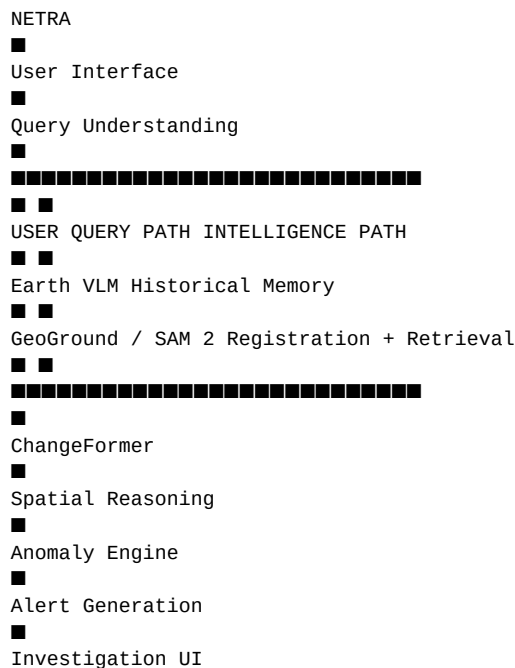
The user can then open an investigation view showing before/after imagery, change masks, affected polygons, coordinates, confidence, sensor evidence, and a natural-language explanation.

5. Key Differentiator

The feature is not simply a conventional image-difference system. Conventional change detection generally takes Image A and Image B and produces a change map. Netra combines temporal change detection with geospatial memory, object-level reasoning, anomaly analysis, multi-sensor verification, and natural-language investigation.

The resulting system moves from **Ask** → **Answer** toward **Observe** → **Remember** → **Compare** → **Detect** → **Alert** → **Investigate**.

6. High-Level System Architecture



7. Component Specifications

7.1 Geospatial Metadata Extraction

For every uploaded scene, extract and persist spatial, temporal, sensor, and image metadata.

Required metadata: latitude/longitude, bounding box or footprint, CRS/projection, pixel size, ground sampling distance, acquisition date/time, satellite/sensor, bands, SAR polarization when applicable, orbit information where available, image dimensions, number of bands, resolution, and cloud information when available.

7.2 Geographic Area Identification

Create a stable **area_id** for each geographic region. The system should determine whether a new image footprint intersects or substantially overlaps an existing area. An area then becomes a container for its temporal observations.

Example: AREA_001 → Observation_001 (Jan 2026) → Observation_002 (Jul 2026) → Observation_003 (Jan 2027).

7.3 Object Extraction and Grounding

Convert image content into geographic objects rather than only text descriptions. Candidate classes include buildings, roads, water bodies, trees, agricultural fields, bridges, airports, vehicles, ships, industrial structures, and construction sites.

Each object should store: object_id, class, confidence, geometry, area, centroid, observation_id, and domain-specific attributes. Geometry should be represented in a spatially usable format such as a polygon.

7.4 Spatial Database

Use PostgreSQL with PostGIS as the recommended prototype spatial database. It provides persistent storage for areas, observations, object geometries, spatial relationships, and change events.

Recommended data model

Entity	Important fields
AREA	area_id, geometry, centroid, location, metadata
OBSERVATION	observation_id, area_id, acquisition_date, sensor, resolution, image_path
OBJECT	object_id, observation_id, class, geometry, confidence, attributes
CHANGE_EVENT	change_id, area_id, old_observation, new_observation, geometry, change_type, confidence, severity
ALERT	alert_id, change_id, alert_type, severity, status, reviewed, user_feedback

7.5 Image Co-Registration

Before temporal comparison, corresponding geographic locations must be aligned. Co-registration reduces false changes caused by spatial shifts between observations. The pipeline should account for projection, resolution, image footprint, and sensor-specific characteristics.

7.6 Change Detection

Use ChangeFormer as the primary learned change-detection component in the proposed architecture. Its output is a spatial change map indicating where meaningful visual change is likely to have occurred.

Change detection answers **where** change occurred. Subsequent object-level analysis should answer **what** changed.

7.7 Object-Level Change Detection

For a target object class such as buildings, detect/segment objects independently in the old and new observations, convert them into geographic polygons, and spatially match the polygons.

Intersection over Union (IoU) can be used as a basic matching metric: $\text{IoU} = \text{Area}(\mathbf{A} \cap \mathbf{B}) / \text{Area}(\mathbf{A} \cup \mathbf{B})$. High overlap suggests the same object; low or absent overlap suggests a new, removed, or substantially modified object.

Object-level outcomes should include: **NEW, REMOVED, UNCHANGED, MODIFIED**.

7.8 Change Categories

Category	Example transition
Construction	No building → building
Demolition	Building → no building
Expansion	Small footprint → larger footprint
Road development	Existing road → expanded/new road
Vegetation loss	Vegetation → built-up/bare land
Water change	Dry land → water, or water → dry land

7.9 Anomaly Engine

Not every detected change should produce an alarm. The anomaly engine evaluates the magnitude, type, confidence, and historical context of a change.

For the first prototype, use a transparent rule-based engine. Example rules include: alert when new buildings exceed a configurable count; alert when vegetation loss exceeds a configured percentage; alert when water-area change exceeds a configured percentage; increase severity when changes persist across multiple observations.

A later research version can learn a baseline from multiple historical observations and assign an anomaly score based on deviation from normal behavior.

7.10 Historical Baseline

As observations accumulate, Netra can estimate normal temporal behavior. For example, if a region historically adds approximately 0–2 buildings per observation interval, a sudden increase of dozens of buildings can be flagged as statistically unusual.

7.11 Multi-Sensor Verification

Use optical and SAR observations as complementary evidence. A potential structural change identified in optical imagery can be cross-checked against Sentinel-1 SAR change evidence. Agreement between independent sensors can increase confidence and reduce false positives caused by clouds, shadows, illumination, or sensor-specific appearance.

7.12 Alert Generation

Alerts should be actionable and evidence-based. A recommended alert contains: area, observation interval, change type, number of affected objects, affected area, confidence, severity, optical evidence, SAR evidence when available, and a link to the investigation view.

Example alert

■ POTENTIAL URBAN DEVELOPMENT

Area: AREA_001
Observation interval: January 2026 → July 2026
Change: 6 new building structures
Affected area: 4,820 m²
Optical evidence: Detected
SAR evidence: Detected
Confidence: 91%
Severity: HIGH

8. Investigation Layer

Every alert should provide a human-in-the-loop investigation workflow. Netra should not automatically assert that an anomaly represents a specific real-world cause with certainty. Instead, it should expose evidence and allow the user to verify, reject, or investigate the event.

Investigation view should contain: before image, after image, change mask, highlighted object polygons, coordinates, object counts, affected area, confidence, temporal interval, sensor information, and available cross-sensor evidence.

Recommended actions: **Investigate Change**, **View Coordinates**, **Compare SAR**, **Ask Netra**, **Research Further**, **Confirm**, and **False Alarm**.

9. Natural-Language Investigation

The VLM should be used after structured geospatial evidence has been produced. Provide the model with the before/after imagery, change mask, object polygons, coordinates, historical statistics, and sensor metadata.

This enables questions such as: “What changed?”, “How many buildings were constructed?”, “Where are the new structures?”, “What is the approximate affected area?”, “Did vegetation decrease?”, and “Does SAR imagery support this change?”

This evidence-first design reduces dependence on free-form visual guessing and gives the language model structured spatial context.

10. False Positive Management

False alarms are a major engineering challenge. The system must explicitly handle clouds, shadows, seasonal vegetation, image misalignment, different acquisition conditions, SAR speckle, sensor differences, and resolution changes.

Recommended mitigations include cloud masking, careful co-registration, resolution normalization, temporal filtering, object-level matching, confidence thresholds, and multi-sensor verification.

11. Recommended Technology Stack

Layer	Recommended technology
Frontend	React / Next.js
Backend/API	FastAPI
Remote-sensing VLM	Earth-OneVision-class model
Adaptation	LoRA / QLoRA
Grounding	GeoGround
Segmentation	SAM 2
Change detection	ChangeFormer
Satellite sources	Sentinel-1 and Sentinel-2
Spatial database	PostgreSQL + PostGIS
Raster processing	GDAL, Rasterio
Vector processing	GeoPandas, Shapely
Map visualization	Leaflet or MapLibre
Background jobs	Celery + Redis (optional)

12. Development Roadmap

Phase 1 — Geospatial Foundation: upload image → extract metadata → identify AOI → store image and metadata in PostGIS.

Phase 2 — Object Intelligence: implement building, water, road, and vegetation detection/segmentation; generate polygons, bounding boxes, coordinates, counts, and areas.

Phase 3 — Historical Memory: create AREA_ID and observation history; support retrieval of previous observations for the same geographic area.

Phase 4 — Change Detection: integrate ChangeFormer and produce a before/after change mask.

Phase 5 — Object-Level Change: spatially match old/new object polygons and classify new, removed, unchanged, and modified objects.

Phase 6 — Anomaly Detection: introduce transparent rules and configurable thresholds; generate severity and confidence scores.

Phase 7 — Multi-Sensor Verification: add optical + SAR cross-checking to improve confidence.

Phase 8 — Investigation: connect alerts to an evidence-rich UI and VLM-powered natural-language investigation.

13. Recommended MVP

For the first working demonstration, do not attempt every object type or a fully learned anomaly model. Build one strong vertical slice around **building-change detection**.

MVP pipeline:

- Upload Scene A
 - Extract geospatial metadata
 - Detect/segment buildings
 - Store building polygons in PostGIS
- Upload Scene B of the same area
 - Register scenes
 - Detect/segment buildings again
 - Spatially match polygons
 - Identify NEW/REMOVED/MODIFIED buildings
 - Calculate change magnitude
 - Apply anomaly rules
 - Generate alert
 - Show before/after + change mask + map
 - Allow user verification

14. GPU Considerations

For a 16 GB RTX 3060-class environment, avoid keeping all heavy models resident on the GPU simultaneously. Use sequential inference, memory-aware model loading, mixed precision where supported, tiling for large imagery, and lightweight CPU-side geospatial operations.

The first prototype can be assembled without training every component from scratch. Start with pretrained models and integrate the complete data flow before investing in fine-tuning.

15. Success Criteria

Criterion	Expected behavior
Geospatial identity	Same geographic area can be recognized across observations.
Historical memory	Previous observations and detected objects are retrievable.
Object grounding	Detected objects have usable geographic geometries.

Criterion	Expected behavior
Temporal comparison	Old and new observations can be aligned and compared.
Change classification	New, removed, unchanged, and modified objects can be identified.
Anomaly alert	Significant changes trigger an actionable alert.
Evidence	Alert provides before/after imagery and spatial evidence.
Human verification	User can confirm or reject the alert.
Investigation	User can ask follow-up questions about the detected event.

16. Final Product Vision

The long-term goal is for Netra to maintain a persistent, temporal digital representation of geographic areas. Users should be able to ask questions about an area at any time while the system independently maintains awareness of how that area is changing.

Netra should evolve from a satellite-image question-answering system into a persistent Earth-observation intelligence system.

Product statement:

“Netra doesn’t just answer questions about Earth. It remembers what Earth looked like, detects when it changes, and brings important changes to the user’s attention.”